



Delhi-Metro-DMRC-Commuter-Density-Bottleneck-Analytics

Author: Aditya Sharma

Tools Used: Google BigQuery (SQL), Python (Pandas, NumPy, SciPy, Matplotlib, Seaborn), SQLite, Google Data Studio

Dataset: 150,000 Transacted Metro Trip Records

1. ASK PHASE (Business Problem & Objectives)

1.1 Background & Context

The Delhi Metro system serves millions of daily commuters across the National Capital Region (NCR). As transit demand grows, transit authorities face complex operational trade-offs between managing station congestion, optimizing multi-product ticket revenue, and scheduling maintenance without disrupting peak traffic.

1.2 Key Stakeholders

- **Executive Leadership:** Operations Director, Chief Revenue Officer (CRO).
- **Operational Teams:** Station Managers, Train Scheduling & Dispatchers.
- **Commuters:** Public transit users requiring reliable, un-congested travel.

1.3 Business Questions

1. **Bottleneck Identification:** Which core interchange stations experience severe commuter congestion, and how can capacity be optimized?
2. **Revenue Monetization:** Which ticketing product (Tourist Card, Single Token, Smart Card) drives maximum revenue yield?
3. **Fare Sensitivity & Pricing:** Is there a statistically significant difference in fares during peak surge hours vs. non-peak hours?
4. **Seasonality:** What seasonal trends exist in ridership, and when should annual maintenance be scheduled?

2. PREPARE PHASE (Data Ingestion, Schema & Validation)

2.1 Data Sources & Storage Architecture

- **Staging Database:** Local SQLite relational database ([delhi_metro_clean.db](#)).
- **Enterprise Warehouse:** Google BigQuery dataset ([delhi-metro-project-503214.Delhi_Metro](#)).
- **Volume:** 150,000 trip transaction records spanning multi-year operations.

2.2 Data Integrity & Validation Constraints

Before processing, automated Python scripts validated data cleanliness and logged ETL metrics into [logs/etl_pipeline.log](#).

- **Validation Rules:** Purged records with negative fares (Fare < 0) or zero/missing passenger counts (Passengers <= 0).
- **Schema Design:** Implemented a BigQuery Star Schema comprising fact tables ([clean_fact_trips](#)) and dimensional features.

3. PROCESS PHASE (SQL Data Cleaning & Feature Engineering)

3.1 SQL Transformation Pipeline

Data was cleaned using BigQuery SQL DDL/DML queries leveraging key SQL operations:

- **TRIM():** Cleaned leading/trailing whitespaces from station names ([From_Station_Clean](#), [To_Station_Clean](#)).
- **COALESCE():** Imputed missing ticket types (defaulted to 'Standard') and null traffic remarks (defaulted to 'regular').
- **CAST():** Standardized numerical datatypes ([Distance_km](#) to [FLOAT64](#), [Passengers](#) to [INT64](#)).
- **CASE Statements:** Dynamically assigned demand surge categories ([High Demand Surge](#), [Special Event Spike](#), [Weekend Moderate](#), [Operational Maintenance](#), [Normal Flow](#)).

SQL

-- Sample BigQuery SQL Cleaning Snippet

```
SELECT DISTINCT
```

```
  TripID,
```

```
  Date,
```

```
  TRIM(From_Station) AS From_Station_Clean,
```

```
  TRIM(To_Station) AS To_Station_Clean,
```

```
  CAST(Distance_km AS FLOAT64) AS Distance_Float,
```

```
  CAST(Fare AS FLOAT64) AS Fare_Float,
```

```
  COALESCE(TRIM(Ticket_Type), 'Standard') AS Ticket_Type_Clean,
```

```
  CASE
```

```
    WHEN LOWER(Remarks) LIKE '%peak%' THEN 'High Demand Surge'
```

```
    WHEN LOWER(Remarks) LIKE '%festival%' THEN 'Special Event Spike'
```

```
    ELSE 'Normal Flow'
```

```
  END AS Demand_Category
```

```
FROM `delhi-metro-project-503214.Delhi_Metro.clean_fact_trips`;
```

3.2 Engineered Features

To prepare the dataset for deep analytics, 5 domain-specific features were added:

1. $\text{Revenue_Per_KM} = \frac{\text{Total_Revenue}}{\text{Distance_KM}}$
2. `Passenger_Density_Group ( High Congestion Peak ≥ 30, Medium , Low Volume )`
3. `Is_Peak_Surge` (Binary Flag: 1 for Surge/Events, 0 for Off-Peak)
4. `Fare_Tier ( Budget <₹50 , Standard ₹50–100 , Premium >₹100 )`
5. `Revenue_Impact_Class ( High Revenue Trip ≥ ₹2500)`

4. ANALYZE PHASE (Advanced SQL, Python EDA & Hypothesis Testing)

4.1 Advanced BigQuery SQL Analytics

- **CTE & HAVING Query:** Isolated major bottleneck hubs handling \$>5,000\$ trips. **Rajiv Chowk (349,784 passengers)** and **Noida City Centre (278,174 passengers)** emerged as primary hubs.
- **Window Functions & Ranking (`DENSE_RANK()`):** Ranked top origin-destination pairs per demand category (e.g., **Rajiv Chowk -> Aiims** leads High Demand Surge with 5,538 passengers).
- **Subquery Analysis:** Calculated revenue contribution percentage per ticket type.

4.2 Statistical Hypothesis Testing (Python SciPy)

Test Conducted	Hypothesis / Test Metric	Statistical Result	Business Finding
Two-Sample T-Test	$H_0: \mu_{\text{Peak}} = \mu_{\text{Non-Peak}}$	$t = 0.6480,$ $p = 0.5170$	Fail to Reject H_0. Current fares during peak hours (Mean: ₹105.21) do not differ from non-peak hours (Mean: ₹105.03). High opportunity for dynamic time-of-day tariffs.

95% Confidence Interval	Population Mean Fare	Mean = ₹105.12, CI = ₹104.84, ₹105.39	Extremely low margin of error across 150,000 trips.
Pearson Correlation	Fare_INR vs Total_Revenue	r = 0.9011, p = 0.0000	Statistically Significant (p < 0.05). Strong direct linear relationship driving overall revenue.
Chi-Square Test	Ticket Type vs. Demand Category	X ² = 25.1968, p = 0.0664	Fail to Reject H₀ (p > 0.05). Ticket choice selection is independent of demand surge scenarios.

5. SHARE PHASE (Visual Analytics & Looker Studio Dashboard)

5.1 Exploratory Data Visualizations

Generated high-resolution analytical plots using Matplotlib and Seaborn:

- **Top Origin Stations Bar Chart:** Highlights extreme traffic concentration at Rajiv Chowk and Noida City Centre.
- **Ticket Type Revenue Donut Chart:** Demonstrates **Tourist Cards driving 39.5% (~₹12.45 Cr)** of total revenue, followed by Single Tokens (25.0%) and Smart Cards (24.8%).
- **Monthly Ridership Line Chart:** Reveals a consistent, recurring **12–14% ridership drop every February (~74,000–77,000 trips)** compared to mid-year peaks (~87,000 trips).

5.2 Interactive Looker Studio Executive Dashboard

An executive-level dashboard was published on Google Looker Studio:

- **Global Controls:** Dynamic drop-down filters for **Year/Month**, **Demand_Category**, **Ticket_Type**, and **Origin_Station**.

- **KPI Scorecards:** Total Revenue (₹315.4M / ₹31.53 Cr), Total Commuters (3.0M), Total Trips (150,000), Average Fare (₹105.12).
- **Deep-Dive Matrix:** Pivot Heatmap analyzing Demand_Category vs. Fare_Tier alongside a Bottleneck Route Data Table.

6. ACT PHASE (Strategic Recommendations & Action Plan)

EXECUTION ROADMAP & TIMELINE MATRIX			
1. Short-Term (0 - 3 Months)	2. Mid-Term (3 - 6 Months)	3. Long-Term (6 - 12 Months)	
Flow Control & Digital Passes	Dynamic Pricing & Overhauls	Network & Fleet Expansion	

6.1 Short-Term Execution (0 – 3 Months)

- **Automated Crowd Barriers at Rajiv Chowk & Noida CC:** Deploy dynamic queue gates and dedicated exit corridors during morning (8–10 AM) and evening (5–8 PM) surge windows to reduce station platform dwell times.
- **Digital QR Migration Drive:** Single Journey Tokens generate 25% of revenue but cause long physical ticketing queues. Launch a 10% instant discount campaign for commuters switching from physical tokens to mobile/app QR passes to reduce ticketing booth operational costs.

6.2 Mid-Term Execution (3 – 6 Months)

- **Time-of-Day (ToD) Dynamic Tariffs:** Capitalize on the static pricing gap proven by the T-Test ($p = 0.517$). Introduce a **15% mid-day off-peak discount (11 AM – 4 PM)** to flatten the rush-hour commuting curve and incentivize flexible travel.
- **Designated February Maintenance Window:** Align annual heavy rail overhauls, signaling maintenance, and track upgrades exclusively during February to leverage the recurring 12–14% seasonal ridership drop without disrupting peak festive transit.

6.3 Long-Term Execution (6 – 12 Months)

- **Optimized Fleet Headways:** Reduce train arrival intervals from 2.5 minutes to 90 seconds on critical bottleneck corridors (Rajiv Chowk -> AIMS, Rajiv Chowk -> Laxmi Nagar) during peak surge windows.
- **Feeder Network Partnerships:** Partner with last-mile electric mobility providers at high-volume origin stations (Hauz Khas, Janakpuri West, Dilshad Garden) to capture high-margin, short-distance commuter trips.